

ActFloor-GAN: Activity-Guided Adversarial Networks for Human-Centric Floorplan Design

Shidong Wang[✉], Wei Zeng[✉], *Member, IEEE*, Xi Chen[✉], Yu Ye,
Yu Qiao[✉], *Senior Member, IEEE*, and Chi-Wing Fu[✉], *Member, IEEE*

Abstract—We present a novel two-stage approach for automated floorplan design in residential buildings with a given exterior wall boundary. Our approach has the unique advantage of being human-centric, that is, the generated floorplans can be geometrically plausible, as well as topologically reasonable to enhance resident interaction with the environment. From the input boundary, we first synthesize a human-activity map that reflects both the spatial configuration and human-environment interaction in an architectural space. We propose to produce the human-activity map either automatically by a pre-trained generative adversarial network (GAN) model, or semi-automatically by synthesizing it with user manipulation of the furniture. Second, we feed the human-activity map into our deep framework *ActFloor-GAN* to guide a pixel-wise prediction of room types. We adopt a re-formulated cycle-consistency constraint in *ActFloor-GAN* to maximize the overall prediction performance, so that we can produce high-quality room layouts that are readily convertible to vectorized floorplans. Experimental results show several benefits of our approach. First, a quantitative comparison with prior methods shows superior performance of leveraging the human-activity map in predicting piecewise room types. Second, a subjective evaluation by architects shows that our results have compelling quality as professionally-designed floorplans and much better than those generated by existing methods in terms of the room layout topology. Last, our approach allows manipulating the furniture placement, considers the human activities in the environment, and enables the incorporation of user-design preferences.

Index Terms—Floorplan design, room layout, human-centric, GAN

1 INTRODUCTION

COMPUTATIONAL architecture design attempts to automate the generation of architectural layouts that define the spatial configuration of rooms, walls, and doors (Fig. 1c) within a given exterior wall boundary (Fig. 1a). Automated floorplan generation for residential buildings has been popular for decades [27]. The topic has attracted researchers not only in architecture (e.g., [1], [37], [47]), but also in related fields like computer graphics (e.g., [2], [36], [50]). In general,

a two-stage approach is adopted: first collect a set of constraints related to the geometric (e.g., room sizes and positions) and/or topological (e.g., room adjacency) properties of the room layout [37], then optimize to produce a room layout that fulfills the constraints.

Conventional methods exploit various design constraints and optimization techniques. However, there still remain many challenges. First, floorplan design tends to be ill-defined and over-constrained [1]. Common geometric and topological properties can be modeled relatively easily by means of objectives such as cost and performance. However, architectural designers are also concerned with the aesthetic and usability aspects of a room layout, which are generally more difficult to describe formally [37]. Second, rooms and walls in a floorplan are deformable shapes that often do not have pre-defined dimensions, thus bringing additional difficulty for the subsequent optimization model to converge. Last but not least, the design space of floorplans is diverse there exist a wide range of feasible room layouts for the same building boundary. Taking Fig. 1a as an example, there exist multiple feasible floorplans (Fig. 1c). Architectural designers often want to explore diverse design options rather than having only one single solution.

Recently, deep learning methods are being increasingly used for automated floorplan design. On the one hand, vast amounts of floorplans in vector-graphics format have been manually curated (e.g., CubiCasa5K [20] and RPLAN dataset [51]) or automatically mined from raster images (e.g., [33], [55]), providing reliable data for neural networks to learn. The floorplans are from real-world designs, which reflect good practices of room layout design by professional designers. On the other hand, various efforts have been put on modeling room layouts into network consumable

- Shidong Wang is with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, and also with the Shandong University, Jinan, Shandong 250100, China. E-mail: sdwang96@gmail.com.
- Wei Zeng is with The Hong Kong University of Science and Technology, Guangzhou, China. E-mail: weizeng@ust.hk.
- Xi Chen is with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, and also with the University of Chinese Academy of Sciences, Beijing 100049, China. E-mail: xi.chen2@siat.ac.cn.
- Yu Ye is with Tongji University, Shanghai 201804, China. E-mail: yye@tongji.edu.cn.
- Yu Qiao is with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China. E-mail: yu.qiao@siat.ac.cn.
- Chi-Wing Fu is with the Chinese University of Hong Kong, Hong Kong SAR, China. E-mail: cwfu@cse.cuhk.edu.hk.

Manuscript received 23 June 2021; revised 15 October 2021; accepted 2 November 2021. Date of publication 9 November 2021; date of current version 31 January 2023.

This work was supported in part by the Fundamental Research Funds for the Central Universities under Grant 22120210540 and in part by the Research Grants Council of the Hong Kong Special Administrative Region under Grant CUHK 14206320.

(Corresponding author: Wei Zeng.)

Recommended for acceptance by Y. Liu.

Digital Object Identifier no. 10.1109/TVCG.2021.3126478

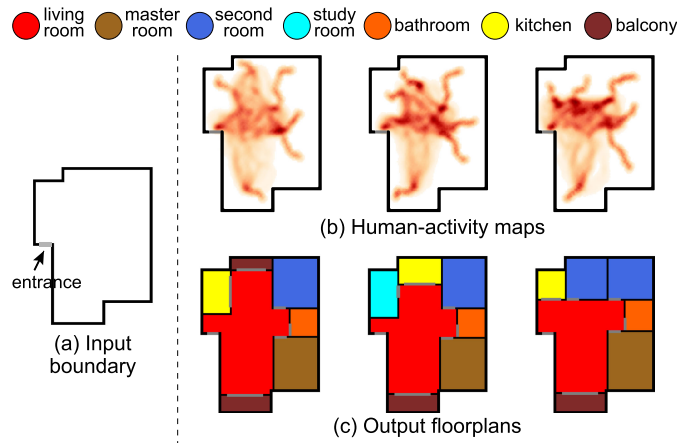


Fig. 1. Given an exterior wall boundary as input (a), our method can generate diverse floorplans (c) that are not only geometrically plausible but also topologically reasonable in correspondence with the human-activity maps (b).

formats, and on designing neural networks to automate floorplan generation. For example, Wu *et al.* [51] developed an iterative prediction model to determine the room connections and positions, and an encoder-decoder network to compute the room sizes and wall locations. Later, HouseGAN [40] and Graph2Plan [16] explored generative networks to learn room relations. However, these deep-learning-based methods typically involve network structures that are complicated. Taking the work of Wu *et al.* [51] as an example, their model consists of four deep networks, including a regression network for locating the living room, another regression network for predicting the other room positions and types, a continuing network to determine if the process should continue, and an encoder-decoder network to predict walls. Moreover, architectural designers often want to explore various design options, whilst many existing methods fail to meet the requirements.

Modern architecture is an exploratory process of creating designs for people [34]. Human activities and their behavior preferences are always key concerns at the preliminary phase of the design process [15]. Following the protocol of architecture design, we propose a human-centric approach using human-activity maps, as guidance for floorplan design. Being human-centric, a floorplan should be geometrically plausible and satisfy topological requirements, such that the residents can have pleasant interactions with the environment [1]. As suggested by a collaborating architect, we leverage the concept of human-activity map (see Fig. 1b) that describes human spatial behavior in an architectural space [10], reflecting both the spatial configuration and human-environment interaction of room layouts. Given an input boundary, we first provide two alternative approaches for generating human-activity maps: (i) a fully-automated method via a generative neural network, and (ii) a semi-automatic method via a user interface.

Next, we present a two-stage approach that produces vectorized floorplans from building boundaries and human-activity maps. At the first stage, we design a new deep framework, namely *ActFloor-GAN*, that takes a human-activity map to guide the floorplan generation from the input boundary. All inputs and outputs of *ActFloor-GAN* are raster images, allowing us to adopt a re-formulated cycle-consistency constraint,

including (i) an adversarial loss to encourage the generated floorplans to be indistinguishable from real floorplans; (ii) a cycle-consistency loss to enforce the forward-backward consistency when translating between the boundary and floorplan domains; and (iii) an identity loss to regularize the generator to form a near identity mapping when a real sample of the target domain is provided as an input to the generator. We train *ActFloor-GAN* on the RPLAN dataset [51] and show that the model improves the overall performance in terms of the pixel-wise prediction accuracy compared with prior methods. At the second stage, we convert the pixel-wise predictions from *ActFloor-GAN* into vectorized floorplans for convenient usage by architects. Subjective evaluation by architects shows that the floorplans by our approach have compelling quality as the professionally-designed floorplans, and are more preferred than those by Wu *et al.* [51].

The main contributions of our work are as follows:

- We design a novel deep learning method that employs the human-activity map to guide the floorplan design (Section 5). To the best of our knowledge, this is the first attempt that considers human activities in computational floorplan design, in which we look at both the spatial configuration and human-environment interaction.
- We develop automatic and semi-automatic approaches to synthesizing the human-activity map (Section 4). Experimental results show that the automatic approach produces accurate piece-wise room predictions in comparison to prior methods without guidance by human-activity maps and adoption of cycle-consistency constraints (Section 6.2), and generates vectorized floorplans of high quality that are comparable to professional designs, as evaluated by architects (Section 6.3). Also, the semi-automatic approach enables flexibility to generate diverse human-centric floorplans (Section 6.4).

2 RELATED WORK

Computational Layout Design. Layout design is an important area of inquiry that aims to find a feasible spatial configuration for a set of interrelated objects. Research on computational layout design spans a wide range of scenarios, such as user interface (e.g., [26], [48]), visualization (e.g., [4], [7]), urban planning (e.g., [43], [52]), indoor scene (e.g., [32], [49]), document layout (e.g., [41]), and shape structures (e.g., [29]). In this work, we focus on the computational architecture layout design, specifically to generate room layouts for an input building boundary.

The design space of floorplan can be decomposed into two aspects: *geometry* that constrains the position and size of each room, and *topology* that concerns the logical relationships between rooms [37]. Conventional methods for computational architecture layout design typically adopt a two-stage approach—first determine the relevant constraints, then optimize the layout. Merrell *et al.* [36] sampled the topological relationship of room adjacencies from real-world data, then applied the relationship to optimize room layouts via stochastic optimization. Bao *et al.* [2] characterized local shape variations as constrained optimization, and

linked them to a portal graph to facilitate layout exploration. Wu *et al.* [50] identified high-level constraints of room size, position, and adjacency, and formulated the constraints as a mixed-integer quadratic programming problem.

Though being automatic, the above approaches typically suffer from the limitations of oversimplified constraints and crudely estimated parameters [2]. Architectural design inherently involves subjective decisions such as aesthetics and domain expertise, involving a complex and high-dimensional solution space. Data-driven approaches can suppress the deficiency by learning fine layouts from the existing databases and applying the learned knowledge to generate new layouts. House-GAN [40] builds a relational generative adversarial network that takes a bubble diagram to encode the room adjacency as constraints and generates room layouts that satisfy the constraints. The high-level constraints are extracted from a linguistic expression that describes the house details [6]. Both methods leave to users the selection of the generated boundaries, without specifying the boundary as a constraint that is commonly adopted by architects. Graph2Plan [16] addresses the limitation by associating the boundaries using an adjacency layout graph and retrieving the associations from an existing database. The method, however, requires a detailed structured representation, and the quality of the generated room layouts by the method can be severely affected by the retrieved boundary. Also, the topology graph alone may not sufficiently capture the human spatial behavior and experience [10].

Alternatively, Wu *et al.* [51] directly took a building boundary as input without requiring manual preparation of the topology graph and high-level room adjacency constraints. The method employs an iterative prediction model that utilizes a CNN to first regress the living room center, an encoder-decoder network to iteratively determine the center of each room, and another CNN to control the iterative process. However, the iterative process has a long-running time, as it typically requires many iterations to cease. In this work, we propose a new generative adversarial network (GAN) model that directly predicts piecewise room layouts. More importantly, our model encapsulates the human-activity maps to constrain the occupants' interactions with the environment and render more realistic room layouts. We show the superior performance of our approach over Wu *et al.* [51] in terms of preference by five architects.

Human-Activity Map. In a typical architecture design process, architects first predict functions of interior space to estimate potential human activities, based on which they draw initial bubble charts that reflect the functional arrangement of the space, and finally generate detailed floorplans [12]. That is, human activities are considered at the preliminary phase of the design process for interior architecture design [15], even prior to topology as 'form follows functions' [8]. Many studies leverage bubble charts as guidance for floorplan design (e.g., [16], [36], [40]), yet few attentions have been paid to human activities. This work exploits human-activity map that describes the spatial behavior of residents in an architectural space, and encodes the relationship between the environment (room layouts and furniture locations) and the residents' activities [10]. To a certain extent, human-activity maps reveal users' behavior intensity

in buildings, helping to locate frequently-used walking paths and dwell points. The information is important concerns in architectural design process [23]. Some studies have employed the human-activity map to guide architectural design. For example, Feng *et al.* [9] optimized the human crowd properties of mobility, accessibility, and coziness when designing mid-scale layouts such as shopping malls. However, few works have considered human activities in apartment-scale residential design, and a particular challenge is that human activity is hard-to-quantify [3]. Conventional methods for generating human-activity maps typically rely on time-consuming agent-based crowd simulations, such as [42], [46]. Recently, Qi *et al.* [44] incorporated a human-activity probability map computed from the bi-directional rapidly-exploring random tree (bi-RRT) [24] to generate functional and natural residential rooms. Hu *et al.* [15] designed a neural network to learn environment-crowd relationships from crowd simulations.

This work adopts the human-activity map to guide architecture layout design. Specifically, we offer two approaches to generate human-activity maps from an input boundary: (i) an automatic method with a GAN model trained from synthetic human-activity maps; and (ii) a semi-automatic method using bi-RRT based on user-specified furniture locations. These alternatives enable the generation of diverse floorplan designs to match the architect's preference.

Generative Adversarial Networks. GANs [11] are a class of methods that train deep generative models based on the game-theoretic min-max principles. Basically, the generator G translates a sample from the generator distribution P_G to data distribution P_{data} and the discriminator D determines if a sample belongs to P_G or P_{data} . Typically, we train G and D together, so that both neural networks can improve together. In practice, GANs have been successfully applied in many fields, including image translation (e.g., [17], [28]) and image super-resolution (e.g., [21], [25]).

Recently, GANs have also been employed to facilitate layout design, e.g., content-aware graphic design layout generation [56]. LayoutGAN [30], [31] models the geographic relations of 2D elements and learns to arrange them in layout designs. The method is, however, limited to simple 2D shapes of points, triangles, and rectangles, whilst floorplan design requires arranging more complex polygons within a constrained building boundary. The most related approach to our work is ArchiGAN [5], which directly predicts pixel-wise room types using the *pix2pix* framework [17]. We experimented with the model on the RPLAN dataset [51]. However, its predicted room types are too noisy, so are infeasible for vectorization. This is probably because the mapping from building boundary to realistic floorplan is one-to-many rather than one-to-one. To address the problem, we re-formulate the cycle consistency constraints [57] to improve the piece-wise room predictions and incorporate the human-activity maps in our *ActFloor-GAN* model to guide the floorplan generation.

3 OVERVIEW

Fig. 2 overviews our framework for human-centric floorplan design. It requires only a building boundary as input (Fig. 2a), which is easier to prepare in comparison to high-level constraints, e.g., topology graph of room adjacency,

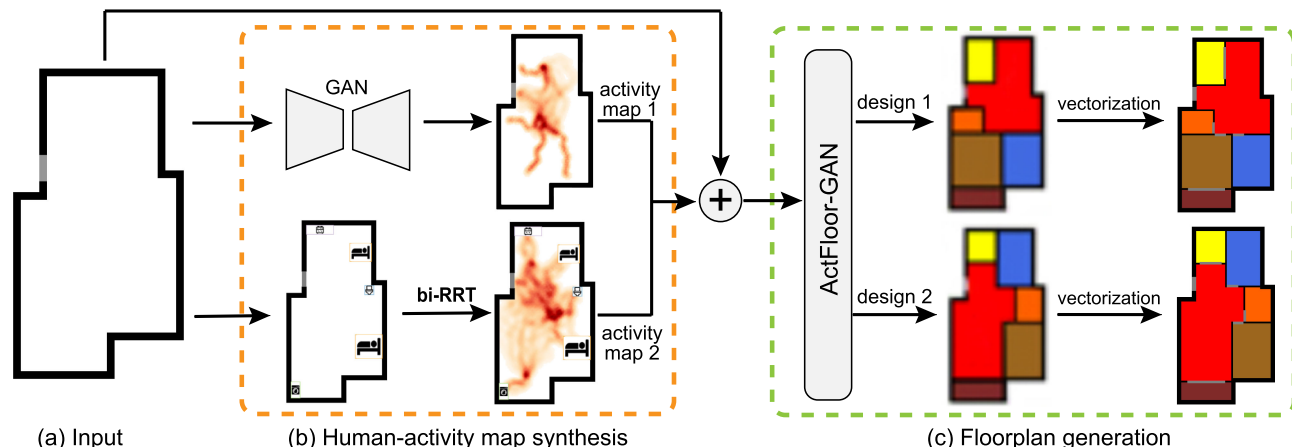


Fig. 2. An overview of our human-centric floorplan design framework. Given an input building boundary (a), we first synthesize the human-activity map (b) using either an end-to-end GAN model (b-top) or bi-RRT [24] based on user manipulation of furniture placement (b-bottom). Next, in the floorplan generation stage (c), our ActFloor-GAN consumes the building boundary & human-activity map, and predicts the piecewise room segmentations that can be vectorized into the final floorplans. ActFloor-GAN can produce diverse designs corresponding to different human-activity maps.

needed in [6], [16], [40], which require domain knowledge from the architects. This work employs the RPLAN dataset [51], which contains 80K floorplans from real-world residential buildings. Each floorplan is represented as a four-channel 256×256 raster image that encodes an *inside mask*, a *boundary mask*, a *room category mask*, and a *room IDs mask*. The room types include the *Living room*, *Master room*, *Second room*, *Study room*, *Bathroom*, *Kitchen*, *Balcony*, and *Other room*. The dataset, however, does not include human activity information, which is crucial in our framework. We opt to synthesize the human-activity maps based on the existing room layouts. As such, we propose a new two-stage approach with the following two major modules.

(i) *Human-activity map synthesis* (Fig. 2b). Well-designed floorplans improve the interaction of the residents with the environment, such as increasing the visibility of an area from observation points [3] and enabling efficient evacuation [15]. This work considers human activities to guide the furniture placement to create functional and natural residential rooms [44]. We incorporate and develop both automatic and semi-automatic methods for generating human-activity maps.

- *Automatic.* We train an end-to-end GAN model by pairing building boundaries from the real-world floorplan dataset RPLAN [51] with corresponding human-activity maps synthesized using bi-RRT [24]. The synthetic human-activity maps consider room-room and room-furniture relations, where the furniture pieces are arranged based on common practices suggested by our collaborating architect. The trained GAN model takes a building boundary as input, and automatically outputs a human-activity map (Fig. 2b-top).
- *Semi-automatic.* We develop an interactive interface for users to arrange furniture, e.g., beds, toilets, and desks, etc., within the building boundary. Our system will then synthesize the corresponding human-activity map using bi-RRT (Fig. 2b-bottom), which can substitute the automatically-generated activity map (Fig. 2b-top) and enhance the flexibility of our approach for on-demand floorplan design.

(ii) *Floorplan generation* (Fig. 2c). Given the building boundary and human-activity maps as inputs, our goal is to

automatically generate diverse floorplans that meet the conditions of human-environment interaction embedded in the activity map. The floorplan generation is a twofold process. First, we train our ActFloor-GAN model to predict pixel-wise room types within the boundary and take the human-activity map to guide the room-type prediction via our re-formulated cycle-consistency constraints. Quantitative experimental results reveal the superior performance of our method in piecewise room-type prediction than prior methods without the human-activity map or the re-formulated cycle-consistency constraints. Next, we use a post-processing module to refine the piecewise room-type predictions into vectorized floorplans. Subjective evaluation by architects shows that the floorplans produced by our approach are comparable with the professionally-designed ones, and better than those by Wu *et al.* [51].

4 HUMAN-ACTIVITY MAP

This section first introduces conventional rules adopted to synthesize the human-activity map from an existing floorplan (Section 4.1). Next, we present how we generate the human-activity maps based on unseen building boundaries in the prediction stage (Section 4.2).

4.1 Synthesis From Room Layout

We utilize the room layout information in the RPLAN dataset [51] to synthesize human-activity maps. As depicted in Fig. 3, the synthesis has two steps: furniture placement (Section 4.1.1) and movement simulation (Section 4.1.2).

4.1.1 Furniture Placement

Human-activity map reflects the residents' interactions with the environment, specifically room layouts and furniture in the rooms. However, the RPLAN dataset contains only the room layout information, but not the furniture locations. To fill the gap, we place furniture in the rooms by carefully considering various requirements. Typically, an apartment contains many different furniture, e.g., bed, and toilet. Detailed furniture configuration can produce fine-grained resident movements. However, excessive furniture in a room could

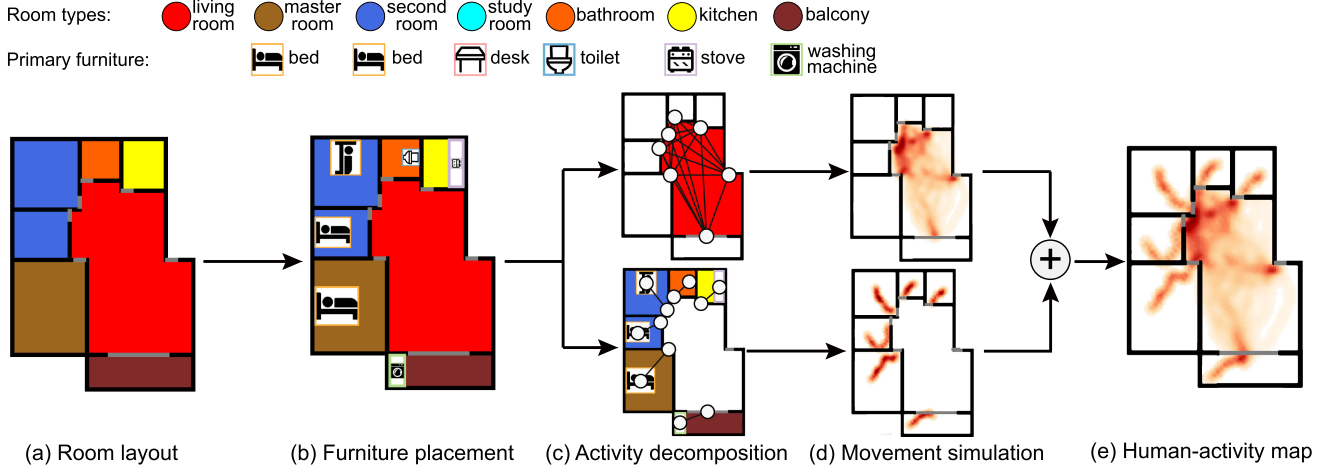


Fig. 3. The synthesis process of the human-activity map. Taking a floorplan from the RPLAN dataset [51] as input (a), we first place primary furniture in each room (excluding the living room) according to conventional rules (b). Next, we decompose the human activities into two parts: in living room and in other rooms (c), and adopt bi-RRT [24] to simulate the resident movements in rooms accordingly (d). Lastly, we combine the simulated movements in different rooms together, yielding a human-activity map (e). Note that the boundary and wall outline in (d) & (e) are for reference only.

over-constrain human activities, and in turn, affect the network’s generalizability in the floorplan generation. We opt to make a trade-off between the movement details and model generalizability, by identifying and arranging primary furniture in each room except the living room.

In consulting our collaborating architect, we formulate the following furniture arrangement rules:

- *Master & Second rooms*: The primary furniture is a *Bed*, of which the size is determined in correspondence with the room size and the position aligns with the wall on the opposite or diagonally opposite side of the room entrance.
- *Study room*: The primary furniture is a *Desk*, of which the size is determined in correspondence with the room size and the position is randomly allocated either beside or opposite to the room entrance.
- *Bathroom*: The primary furniture is a *Toilet*, of which the size is fixed and the position is typically allocated on the opposite or diagonally opposite side of the room entrance.
- *Kitchen*: The primary furniture is a *Stove*, of which the length equals to one of the wall lengths, and the position aligns with the wall opposite or diagonally opposite to the room entrance.
- *Balcony*: The primary furniture is a *Washing machine*, which has a fixed size and is positioned on one of the sides of the balcony. In many Asian cities, from which most floorplans in the RPLAN dataset were collected, it is common to put a washing machine on the balcony.

To enrich the diversity of the synthesized human-activity maps, we provide multiple optional locations for each piece of furniture in a room. Fig. 3b shows an example of furniture placement. Here, the room entrances are retrieved from the RPLAN dataset [51].

4.1.2 Movement Simulation

Human activities in an apartment happen in shared public space (e.g., living room) and also in private space (e.g., master and second room). In a functional residential building, the living room should well-connect the rooms and the main

entrance [37], while furniture arrangement is a key factor that affects the resident movements in other rooms [53]. Hence, this work decomposes the human-activity map into (i) room-room and room-entrance connections in the living room; and (ii) resident movements in the other rooms.

Figs. 3c, 3d depict the procedure of activity decomposition and movement simulation. First, we identify locations of the main entrance of the apartment and entrances to all the other rooms from the floorplans in the RPLAN dataset [51] and automatically connect them all together into a bi-directional graph (Fig. 3c-top). Based on the connectivity graph, we adopt bi-RRT [24] to simulate the resident movements in the living room (Fig. 3d-top) and produce a single-channel image, in which the pixel values indicate the probabilistic human activity intensity, not activity types, in the living room. Similarly, we identify the entrance-furniture connections (Fig. 3c-bottom) and simulate the resident movements in the other rooms (Fig. 3d-bottom). Next, we merge the simulation results in the living room and other rooms together, yielding the final human-activity map (Fig. 3e). We empirically assign weights of 6:4 to the simulation image in the living room and that in the other rooms, since human activities are more frequent in the living room than in the other rooms.

In this way, we can produce a human-activity map for each floorplan in the RPLAN dataset. Fig. 4 presents two example sets of building boundary, floorplan, and human-activity map, where the left two columns are samples from the RPLAN dataset, and the right-most one shows our simulated results. Here, the two boundaries in Fig. 4a are similar, whilst their corresponding floorplans in Fig. 4b are quite different. It is a challenging task for a network to recognize the small difference in the inputs and predicts diverse floorplans as the outputs. We address the challenge by incorporating the human-activity map (see examples in Fig. 4c) as guidance to help the network learn the mapping from building boundary to realistic floorplan.

4.2 Generation From Building Boundary

So far, human-activity maps are synthesized from fine-grained real floorplans, which are available only in the

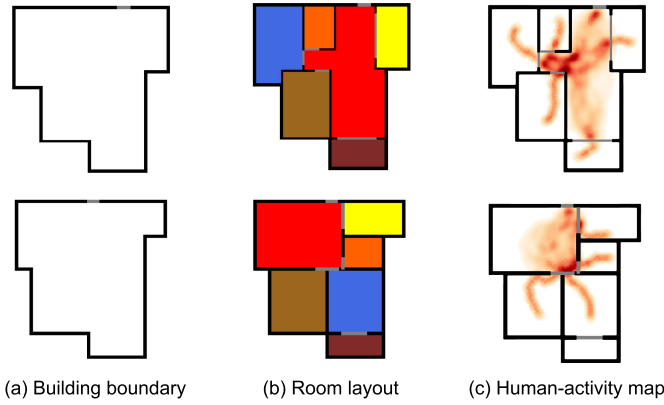


Fig. 4. Sample building boundaries (a) and floorplans (b) from the RPLAN dataset, and the corresponding human-activity maps by our simulation (c).

training stage. Since the test stage also needs human-activity maps to guide the floorplan generation, we thus need to generate human-activity maps from unseen boundaries. We develop two alternative approaches using either an end-to-end GAN model (Section 4.2.1) or a user interface (Section 4.2.2).

4.2.1 Automatic Approach With Generative Network

We train a generative network to automatically generate human-activity maps from input building boundaries. The training set consists of building boundary I_b retrieved from the RPLAN dataset, and human-activity map I_{act} synthesized from the associated floorplan paired with the building boundary. Fig. 5 presents the overall network model, which is based on DCGAN [45].

The generator learns to produce samples of human-activity map $\tilde{I}_{act}$ as the training data. The standard DCGAN generates an image from a sampled latent vector in a uniform distribution. However, this work requires inferring the latent vector from the input building boundary, rather than from a random sample. Hence, we design the generator as an encoder-decoder network, of which the encoder converts the input building boundary I_b into a latent vector, and the decoder maps the latent vector into a sample of human-activity map $\tilde{I}_{act}$. Further, we add skip connections between each layer i in the encoder and layer $n - i$ in the decoder to share low-level information between the encoder and the decoder. Altogether, there are five layers for both the encoder and the decoder. We incorporate noise in the form of dropouts in the first three layers of the decoder at both training and test time, following the same practice as [17], to enable stochastic outputs.

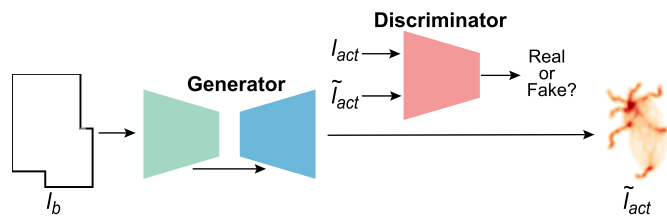


Fig. 5. Architecture of the generative network for automatic human-activity map generation from building boundary.



Fig. 6. Our interface for semi-automatic human-activity map generation includes the *Activity Control Panel* (b) and a *Recommendation Gallery* (c). From the input building boundary (a), our ActFloor-GAN generates a floorplan (d) under the guidance of the human-activity map shown in (b).

The discriminator learns to determine whether a given sample (human-activity map) is produced by a GAN model or synthesized from a real floorplan inside the RPLAN dataset using bi-RRT. The discriminator has five layers to progressively down-sample the input image, similar to the encoder of the generator module. We leverage an adversarial loss that can jointly optimize both the generator and the discriminator simultaneously; the adversarial loss is detailed in Section 5.1. Previous approaches have found it beneficial to mix the GAN objective with a more traditional loss, so we also use an L1 loss as in [17] for robustness.

4.2.2 Semi-Automatic Approach With Interactive Interface

Though being fully automatic, the generative network offers no control over the floorplan generation, failing to offer design options that are preferred by architects. To this end, we develop an interactive interface that allows users to manipulate furniture placement and to generate multiple human-activity maps, thus enabling the generation of diverse floorplans. As shown in Fig. 6, the interface mainly consists of an *Activity Control Panel* (Fig. 6b) and a *Recommendation Gallery* (Fig. 6c).

The *Activity Control Panel* provides two alternative approaches to generate human-activity map: (i) users can choose to use the automatic approach described in Section 4.2.1, by selecting the GAN radio button on the top; and (ii) users can choose to manually manipulate the furniture placement and generate human-activity maps by selecting the User radio button. In the manual mode, users can *add* furniture by dragging furniture icons into the building boundary, *remove* furniture by right-clicking on the icons, or *adjust* the furniture positions by dragging the furniture icons. We synthesize a room entrance for each piece of furniture added into the apartment according to the furniture placement rules described in Section 4.1.1 and the empirical door placement rules described in [51]. Note, room entrances are omitted in the figure, for simplicity. Finally, we synthesize a human-activity map based on the positions of the main entrance, furniture, and room entrances using bi-RRT; see Section 4.1.2 for details.

To alleviate the manual add and drag operations, we retrieve building boundaries from the RPLAN dataset that

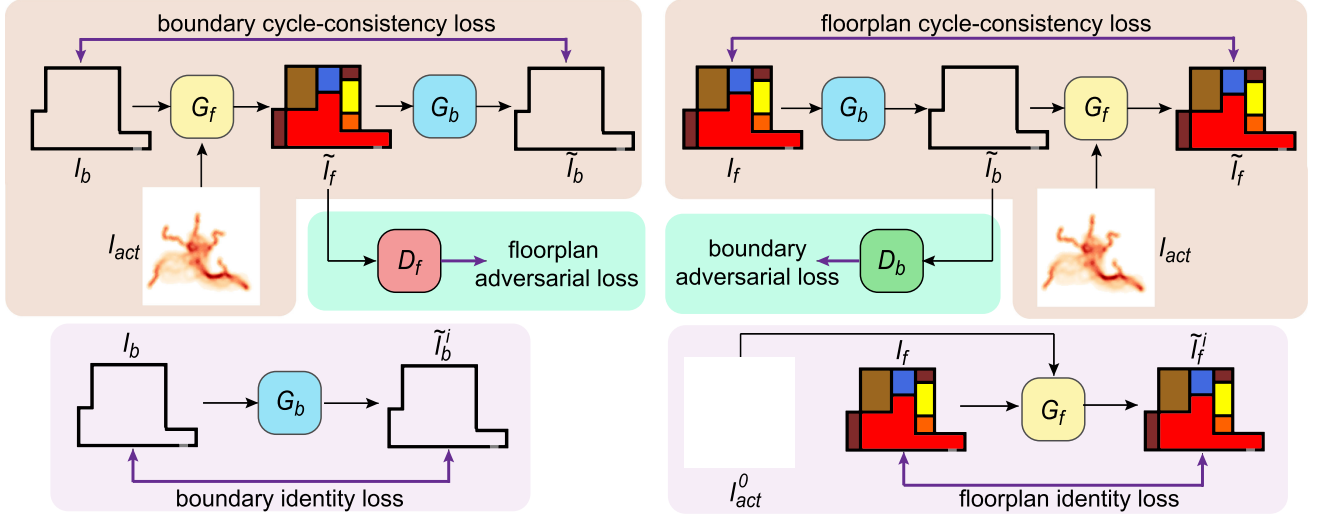


Fig. 7. Our ActFloor-GAN framework consists of two parts: (left) the one to learn from building boundaries, and (right) the other to learn from floorplans. Each part involves three losses: adversarial loss, cycle-consistency loss, and identity loss. G_f and G_b denote the generators that produce floorplan and building boundary, respectively, while D_f and D_b are the discriminators that determine whether the generated images are real floorplan or real building boundary, respectively.

are similar to the input one, and display the top ten most similar ones in the *Recommendation Gallery*. Here, we compute the shape similarity between building boundaries using the Hu moment invariant [38]. Users can select one of the building boundaries in the recommendation list, then apply furniture placement to the input building boundary. Users are allowed to further add/remove furniture, and adjust furniture placement by dragging relevant icons. Besides, the interface also displays the input building boundary (Fig. 6a), and outputs the floorplan (Fig. 6) generated by our ActFloor-GAN under the guidance of the human-activity map presented in Fig. 6b.

With the interface, users can create multiple feasible human-activity maps from the same building boundary. Fig. 14 presents examples of diverse floorplans generated from different human-activity maps on the same boundary.

5 ACTFLOOR-GAN

We design ActFloor-GAN to predict floorplans from building boundaries. Our goal is to learn a mapping function from the domain of building boundaries $P_{data}(I_b)$ to the domain of floorplans $P_{data}(I_f)$, through the guidance of samples from the human-activity map domain $P_{data}(I_{act})$. ActFloor-GAN is a new deep framework that takes an input image $I_b \in P_{data}(I_b)$ that represents the building boundary and a human-activity map $I_{act} \in P_{data}(I_{act})$ to guide the floorplan generation via our re-formulated cycle-consistency constraints. Fig. 7 outlines the overall network architecture of ActFloor-GAN, which has two parts: one to learn from the building boundaries to floorplans $G_f : P_{data}(I_b) \rightarrow P_{data}(I_f)$ (Section 5.1), and the other to learn from the floorplans to building boundaries $G_b : P_{data}(I_f) \rightarrow P_{data}(I_b)$ (Section 5.2).

5.1 Learning From Building Boundary

Fig. 7 (left) shows the part of our framework that learns to generate floorplans from building boundaries. Given an image of boundary I_b , we first use generator G_f to produce a floorplan $\tilde{I}_f$. As discussed above, we aim to produce

multiple floorplans from the same building boundary, by taking human-activity map I_{act} as guidance to constrain the design generation. Hence, generator G_f also takes I_{act} as a part of its inputs, i.e., $\tilde{I}_f = G_f(I_b, I_{act})$. Then, a discriminator D_f is employed to distinguish if $\tilde{I}_f$ is a real floorplan or not, i.e., $D_f(\tilde{I}_f) = \text{real or fake}$? We use a *floorplan adversarial loss* to optimize both generator G_f and discriminator D_f [39]

$$L_{GAN}^b(G_f, D_f) = \mathbb{E}_{I_f \sim P_{data}(I_f)} [\log D_f(I_f)] + \mathbb{E}_{I_b \sim P_{data}(I_b)} [\log (1 - D_f(G_f(I_b, I_{act})))], \quad (1)$$

where $\mathbb{E}$ indicates the error; $P_{data}(I_f)$ and $P_{data}(I_b)$ indicate the data distribution of I_f and I_b , respectively; and $* \sim P_{data}(*)$ indicates that $*$ is selected from the respective dataset $P_{data}(*)$.

To prevent generator G_f from cheating discriminator D_f too easily, we design another generator G_b that produces a boundary image $\tilde{I}_b$ from the generated floorplan image $\tilde{I}_f$, i.e., $\tilde{I}_b = G_b(\tilde{I}_f)$. We leverage a *boundary cycle-consistency loss* to optimize the network [57]

$$L_{cycle}^b(G_f, G_b) = \mathbb{E}_{I_b \sim P_{data}(I_b)} [\|G_b(G_f(I_b, I_{act})) - I_b\|_1], \quad (2)$$

where $\|\cdot\|_1$ denotes the L1 loss of the difference between the two images on corresponding pixel.

Last, we use the real boundary I_b as the input of G_b , and generate a building boundary $\tilde{I}_b^i$ similar to I_b , i.e., $\tilde{I}_b^i = G_b(I_b)$. We would like to force the generated building boundary $\tilde{I}_b^i$ to be the same with I_b , to make sure the generated floorplan is within the same boundary as the input building boundary. We leverage a *boundary identity loss* to achieve the goal

$$L_{identity}^b(G_b) = \mathbb{E}_{I_b \sim P_{data}(I_b)} [\|G_b(I_b) - I_b\|_1]. \quad (3)$$

5.2 Learning From Floorplan

Fig. 7 (right) shows the other part of our framework that learns to generate building boundaries from floorplans. Given a floorplan image I_f , we first use generator G_b to produce a boundary image $\tilde{I}_b$, i.e., $\tilde{I}_b = G_b(I_f)$. Then, we employ discriminator D_b to distinguish if $\tilde{I}_b$ is a real boundary or not, i.e., $D_b(\tilde{I}_b) = \text{real}$ or *fake*? We leverage the *boundary adversarial loss* to supervise the boundary generation in the generative adversarial game

$$L_{GAN}^f(G_b, D_b) = \mathbb{E}_{I_b \sim P_{data}(I_b)} [\log D_b(I_b)] + \mathbb{E}_{I_f \sim P_{data}(I_f)} [\log (1 - D_b(G_b(I_f)))]. \quad (4)$$

Similarly, in order to prevent generator G_b from cheating discriminator D_b too easily, we design another generator G_f whose inputs are the generated boundary image $\tilde{I}_b$ and a human-activity map I_{act} , and output is a floorplan image $\tilde{I}_f$, i.e., $\tilde{I}_f = G_f(\tilde{I}_b, I_{act})$. We leverage *floorplan cycle-consistency loss* to train the network [57]

$$L_{cycle}^f(G_b, G_f) = \mathbb{E}_{I_f \sim P_{data}(I_f)} [\|G_f(G_b(I_f), I_{act}) - I_f\|_1]. \quad (5)$$

Last, we use an empty human-activity map I_{act}^0 with all zero values and the real floorplan I_f as the input to G_f . Ideally, the generated floorplan $\tilde{I}_f$ should be identical to the real floorplan I_f , by minimizing the *floorplan identity loss*

$$L_{identity}^f(G_f) = \mathbb{E}_{I_f \sim P_{data}(I_f)} [\|G_f(I_f, I_{act}^0) - I_f\|_1]. \quad (6)$$

Note that the generator G_b could be trivially replaced by a non-learning method that extracts the boundary of a given floorplan. We keep G_b in the training process, so that the generator G_f can be trained synchronously to better learn the mapping from boundary to floorplan. See Section 6.2 for a comparison with prior methods, especially *Pix2Pix* without using the cycle-consistency constraint, showing that the constraint can help enhance the results.

5.3 Loss Function and Training Strategy

Putting all the losses in the two parts together, the overall loss function for training our ActFloor-GAN model is

$$L_{final}(G_b, G_f, D_b, D_f) = \lambda_1 (L_{GAN}^b(G_b, D_b) + L_{GAN}^f(G_f, D_f)) + \lambda_2 (L_{cycle}^b(G_b, G_b) + L_{cycle}^f(G_b, G_f)) + \lambda_3 (L_{identity}^b(G_b) + L_{identity}^f(G_f)). \quad (7)$$

Following the empirical settings as in [39], we achieved good experimental results by setting weights λ_1 , λ_2 , and λ_3 as 1, 10, and 5, respectively.

We adopt the network architecture designed by Johnson *et al.* [19] as our generator network. Input to generator G_b is a floorplan image in three-channel RGB format, while that for generator G_f is a four-channel grayscale image by concatenating a three-channel building boundary image with a one-channel human-activity map. We use PatchGAN [17] to help discriminators D_b and D_f distinguish whether the image

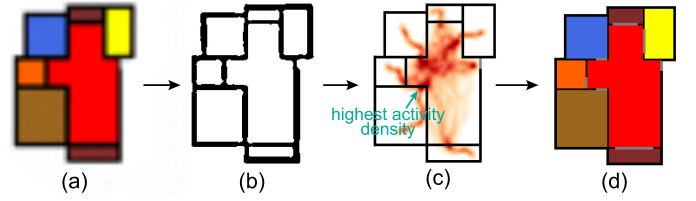


Fig. 8. Illustration of the floorplan vectorization process. (a) Raster image with pixel colors indicating rooms and walls. (b) Noisy exterior and interior walls. (c) Vectorized walls and the corresponding human-activity map. (d) Final vector floorplan with room and door labels.

patches are real or fake. Building boundaries and floorplans are derived from the RPLAN dataset [51], while human-activity maps are synthesized from the floorplans as described in Section 4.1. There are in total 80K pairs of building boundaries, floorplans, and human-activity maps, and we randomly split them into 75k-2.5k-2.5k for training-validation-test sets. We set the batch size to 1 and employed Adam optimizer [22] ($\beta_1 = 0.5$, $\beta_2 = 0.999$) with a fixed learning rate of 0.0002. The network architecture was implemented in PyTorch and ran on a server equipped with an NVIDIA GTX 1080Ti GPU card.

5.4 Floorplan Vectorization

The output of ActFloor-GAN is a three-channel image with pixel colors indicating the room types and positions, as well as the wall positions (Fig. 8a). Further, we leverage a post-processing module to convert the raster image into a vectorized floorplan, to make the results usable by architects. Fig. 8 illustrates the process. From a raster image (Fig. 8a), we first binarize it based on adaptive thresholding to obtain the exterior and interior walls (Fig. 8b). The walls are, however, rather noisy and discontinuous. We further perform morphological closing operations on the results, yielding vertical and horizontal straight lines that form closing boxes. We assign semantics of room types to each room according to the predictions. Next, we find the position with the highest activity density in each room (except for the living room) to position the internal doors. Finally, the vectorized floorplan (Fig. 8d) is generated.

6 EVALUATION AND DISCUSSION

To evaluate the ActFloor-GAN model, we examined the performance of the GAN model for automatic human-activity map generation (Section 6.1), conducted a quantitative comparison between ActFloor-GAN and four prior methods (Section 6.2), and also performed a user evaluation by architects to compare our generated floorplans with those generated by Wu *et al.* [51] and those from the RPLAN dataset (Section 6.3). Then we demonstrate the capability of our approach in generating diverse floorplans (Section 6.4), followed by a discussion in the end (Section 6.5).

6.1 Assessment of Automatic Activity Map Generation

We use mutual information (MI) [35], a concept from the information theory, to assess the GAN model for automatic human-activity map generation. The metric has been applied in medical imaging (e.g., [18]) and data visualization (e.g., [54]). Notice that MI computation requires a preliminary step of image registration, which can be skipped in this work since

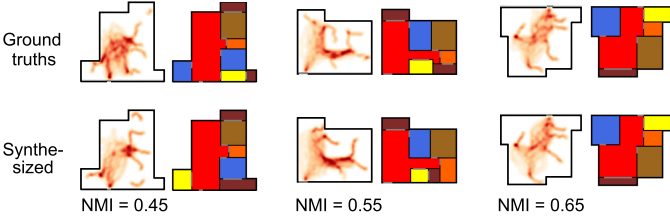


Fig. 9. Visual difference between ground truths (top) and our synthesized human-activity maps (bottom) of the GAN model for automatic human-activity map generation. The corresponding ground-truth and synthetic floorplans are presented besides, while the NMI values are at the bottom, showing that quality of the synthetic floorplans are highly dependent on the similarity of the human-activity map.

the input and output images of the GAN model are naturally aligned. Also, notice that the human-activity maps can be seen as discrete random variables that take intensity values in the range $[0, 255]$. As such, we can measure MI between a ground-truth human-activity map I_{act} and a synthesized one $\tilde{I}_{act}$ as

$$MI(I_{act}, \tilde{I}_{act}) = \sum_{x \in h(I_{act})} \sum_{y \in h(\tilde{I}_{act})} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right), \quad (8)$$

where $h(\cdot)$ denotes the normalized intensity histogram of an image, $p(\cdot)$ is the marginal probability distribution function, and $p(x, y)$ is the joint probability function of the intensity histograms. To eliminate the impacts of image size, we use a normalized version of MI (denoted as NMI)

$$NMI(I_{act}, \tilde{I}_{act}) = \frac{2MI(I_{act}, \tilde{I}_{act})}{H(I_{act}) + H(\tilde{I}_{act})}, \quad (9)$$

where $H(I_{act}) = -\sum_{x \in h(I_{act})} p(x) \log(p(x))$ measures the entropy of the normalized intensity histogram. NMI ranges in $[0, 1]$, where 0 indicates no mutual information while 1 indicates perfect correlation.

We train the GAN model with 75k pairs of building boundaries and simulated human-activity maps, and evaluate the performance using 2.5k testing pairs. The average NMI value is 0.565, and the minimum and maximum values are 0.434 and 0.669, respectively. Fig. 9 presents some examples of ground-truth and synthesized human-activity maps, sorted by their NMI values in ascending order. Notice that the ground-truth and synthesized activity maps look more similar for larger NMI values. The synthetic floorplans guided by the human-activity maps are presented on the right sides correspondingly. It is also observed that the floorplans become more similar when the NMI value is large. For the floorplans shown on the left ($NMI = 0.45$), the number of rooms and room types of the synthesized floorplans are different than those of the ground truths. In contrast, for the floorplans on the right ($NMI = 0.65$), the topological and geographical properties of the synthesized floorplans are identical to those of the ground truths.

6.2 Comparison With Prior Methods

The quality of the generated floorplans depends much on the network predictions. It is necessary to evaluate the prediction results with prior methods that can predict pixel-wise room types from input boundaries. Here, we omit *Wu*

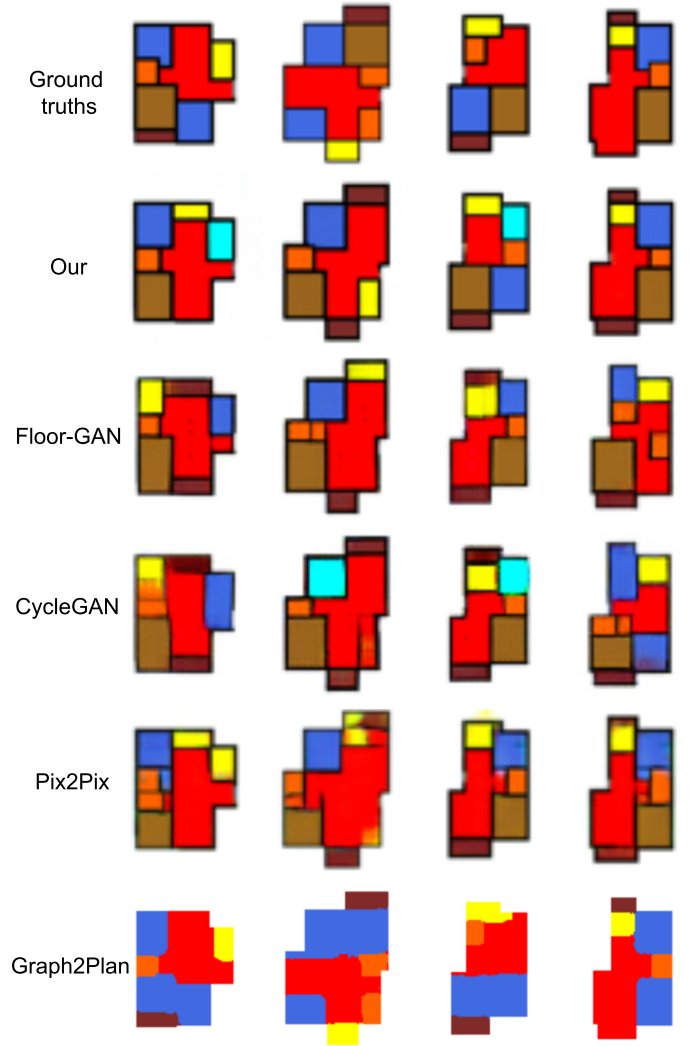


Fig. 10. Visual comparisons with competitors. From top to bottom: ground truths, predictions by *Our*, *Floor-GAN*, *CycleGAN* [57], *Pix2Pix* [17], and *Graph2Plan* [16].

et al. [51], which predicts room centers instead of piecewise room types, and other approaches (e.g., [40]) that require additional inputs such as room adjacency relations.

Prior methods. Some attempts, e.g., *ArchigAN* [5], directly predict pixel-wise room types using *Pix2Pix* [17]. Also, *CycleGAN* [57], which was originally designed for general image-to-image translation, can also be employed to predict pixel-wise room types. We also compare with a network architecture that is the same as *ActFloor-GAN* but without guidance by the human-activity maps, denoted as *Floor-GAN*. We further compare with *Graph2Plan* [16] that predicts room boxes and a raster image of room types, using the ground-truth room graph as guidance. We adopted author-provided implementations and settings and re-trained the models on the RPLAN dataset. Since the prior methods are fully automatic, we utilize automatically-generated human-activity maps to guide *ActFloor-GAN* (denoted as *Our*).

Qualitative comparison. Fig. 10 presents visual comparisons with the competitors. The top row presents ground truths (denoted as *GT*) from the RPLAN dataset, and the other rows present results by *Our*, *Floor-GAN*, *CycleGAN*, *Pix2Pix*, and *Graph2Plan*, respectively. For fairness, all results presented here are intermediate raster images without vectorization.

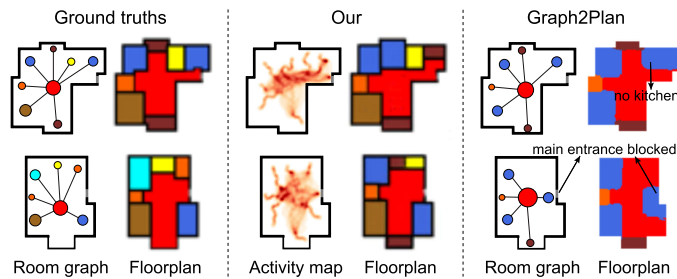


Fig. 11. Comparisons of *Our* guided by automatically synthesized human-activity maps, and *Graph2Plan* guided by room graphs of the most similar floorplans as the input boundaries retrieved from the dataset.

Overall, *Our* model produces the most similar results with *GT*, whilst *Pix2Pix* produces noisy results, and *CycleGAN* and *Floor-GAN* produce grid rooms but the resulting room types differ from *GT*. Here, the *Pix2Pix* model employs the fewest losses in comparison with the other models, yielding non-closing rooms and unqualified predictions for the vectorization module. *CycleGAN* recruits an additional cycle-consistency loss and *Floor-GAN* further adds the identity loss, to constrain the network, thus their predictions are finer with the grid rooms. However, it is difficult for the models to distinguish similar building boundaries. For example, the input building boundaries in columns 3 and 4 are very similar, and the *Floor-GAN* result in column 3 is very similar to the *GT* in column 4. This indicates that cycle-consistency and identity losses alone are susceptible to adversarial perturbations of input boundaries. In contrast, *Our* model can recognize the difference, benefiting from the guidance of automatically generated human-activity maps. *Graph2Plan* also produces fine results, but the method simplifies the prediction by categorizing the *master room* (dark golden in *GT*), *second room* (blue in *GT*), and *study room* (cyan in *GT*) as *bedroom* (blue in *Graph2Plan*). Additional information is needed to distinguish these room types. Notice here that *Graph2Plan* is guided by the ground-truth room graphs. We observe some improper floorplans may be generated, e.g., missing kitchen (Fig. 11 (top-right)) or main entrance blocked by non-living rooms (Fig. 11 (bottom-right)), using room graphs of the most similar floorplans as the input boundaries retrieved from the dataset.

Quantitative comparison. We further conduct quantitative comparisons with the prior models, in terms of both the prediction accuracy and vectorization success rate. Table 1 presents the quantitative comparison results.

- We compute MSE and MAE between the ground truths and the results by *Pix2Pix*, *CycleGAN*, *Floor-GAN*, *Graph2Plan*, and *Our*. Smaller MSE and MAE values indicate better performances. Table 1 shows that *Our* model achieves better performance than *Pix2Pix*, *CycleGAN*, and *Floor-GAN* that predict the same number of room types as *Our*, as both our MSE and MAE values are the smallest. *Graph2Plan* achieves the smallest MAE but a higher MSE than *Our*. We admit that the floorplan generation task may not have a unique floorplan solution for the same boundary, so the use of the ground truths in this experiment is mainly to show the effectiveness of our framework over the prior methods in terms of capturing the latent geometry and topology properties of room layouts.

TABLE 1
Quantitative Comparison With *Pix2Pix*, *CycleGAN*, *Floor-GAN*, and *Graph2Plan* in Terms of Pixel-Wise Accuracy (MSE and MAE), and Vectorization Success Rate

	<i>Pix2Pix</i>	<i>CycleGAN</i>	<i>Floor-GAN</i>	<i>Graph2Plan</i>	<i>Our</i>
MSE	0.569	0.183	0.176	0.238	0.111
MAE	0.349	0.166	0.160	0.079	0.096
Vector.	17/100	25/100	27/100	100/100	83/100

- We further check whether pixel-wise predictions by *Our* and by the prior models can be successfully vectorized (see Section 5.4). A vectorization is regarded as a success if the following conditions are met: (i) every room must form a closed polygon and pixels within each room must have a dominant room type; (ii) the number of room types is balanced, e.g., at least one living room and one master room; and (iii) the main entrance and internal doors (except bathroom and balcony) are directly connected to the living room. We randomly select 100 boundaries and corresponding predictions from *Our* and prior models, and measure their vectorization success rates as presented in Table 1. *Our* model can successfully vectorize 83 out of 100 predictions, which is significantly higher than *Pix2Pix*, *CycleGAN*, and *Floor-GAN*. *Floor-GAN* succeeds only 27 times, which however is still higher than the other two models. *Graph2Plan* succeeds 100 times using predicted room boxes, since it adopts a different vectorization strategy from *Our* and others. The method requires additional networks for predicting room boxes, whilst *Our* model is lightweight.

6.3 Subjective Evaluation by Architects

Next, we explore whether our generated floorplans satisfy the requirements of architects. In consultation with our collaborating architect, we formulate five questions on the quality of floorplan design, as listed in Table 2. Here, *Q1* - *Q2* concern geometry, *Q3* - *Q4* reflects topology, and *Q5* is the overall design. We compare the floorplans by our method with those from the RPLAN dataset that are real-world floorplans, and those by *Wu et al.* [51] that require only the building boundary as the same with our approach.

Participants. We recruited five architects as participants in this study. All are graduate students holding a bachelor's degree in architecture. They all have practical experience in designing floorplans for residential buildings.

Procedure. We randomly selected 30 paired floorplans from the ground truths (*GT*), the results by *Our* model and also the results by *Wu et al.* We then form two sets of pairwise comparisons, *Our* versus *GT*, and *Our* versus *Wu et al.* Examples of *Our* versus *GT* are presented in Fig. 16. We then asked the participants to make a comparison by giving a rating of "better/worse/equally-good" for each pair of floorplans against *Q1* - *Q5*. Notice that the participants were not aware of the origin of the floorplan: *GT*, *Our*, or *Wu et al.*

Result. We collected a total of 1500 answers (2 sets $\times$ 30 pairs $\times$ 5 participants $\times$ 5 questions). For each set of comparison, we employed the Elo rating system to calculate the ratings for all five questions of the two players, i.e., *Our*

TABLE 2
Our Questions Focus on Geometry (Q1 - Q2), Topology (Q3 - Q4), and Overall Design (Q5) of Floorplans

Q1	Which one has better designed <i>room sizes</i> ?
Q2	Which one has better designed <i>room shapes and aspect ratios</i> ?
Q3	Which one has better designed <i>room orientations</i> ?
Q4	Which one has better designed <i>room connections</i> ?
Q5	Overall, which one is better designed?

versus *GT*, and *Our* versus *Wu et al.* The system predicts the outcome of a match between the two players by using an expected score formula, e.g., a player whose rating is 100 points greater than their opponent's is expected to win 64% of the time. If Player *A* has a rating of R_A and Player *B* has a rating of R_B , the expected scores for Players *A* and *B* are

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}, E_B = \frac{1}{1 + 10^{(R_A - R_B)/400}}. \quad (10)$$

Suppose Player *A* was expected to score E_A points but actually scored S_A points, the formula for updating its rating $\tilde{R}_A$ is

$$\tilde{R}_A = R_A + K(S_A - E_A), \quad (11)$$

where K is the maximum possible adjustment per game. In this work, we set a constant value for K as the number of players multiplied by 42, i.e., 84. For each question, a player is assigned an initial rating of 1000. Every time a game is played, the change in the Elo rating of the players depends on the outcome and the expected outcome. A win ("better") counts a score of 1, loss ("worse") counts a score of 0, and draw ("equally-good") counts a score of 0.5.

Fig. 12 presents the Elo ratings of *Our* versus *Wu et al.* on the left, and *Our* versus *GT* on the right, for Q1 - Q5. From Fig. 12 (left), our approach (*Our*) produces much better floorplans than *Wu et al.* in terms of topology-related criteria, i.e., *room orientation* (Q3) and *room connection* (Q4), and slightly better results than *Wu et al.* in terms of geometry-related criteria, i.e., *room size* (Q1) and *room shape and aspect ratio* (Q2). The reason is probably that *Wu et al.* focuses on learning the geometry properties, specifically the center of room positions, in the RPLAN dataset, whilst neglecting the topological relations among rooms. In contrast, the room topology information is embedded in human-activity maps, so ActFloor-GAN can effectively infer the topology information. The difference promotes more preference for our results in terms of the *overall design* (Q5). From Fig. 12 (right), our

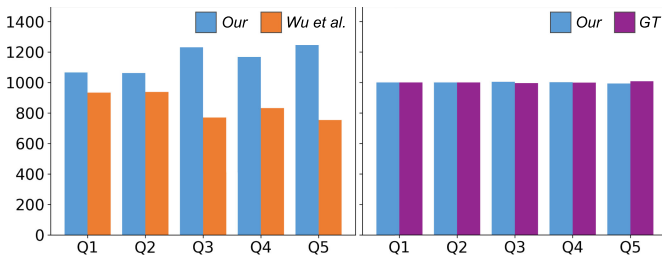


Fig. 12. Elo ratings of *Our* versus *Wu et al.* (left), and *Our* versus *GT* (right) for Q1 - Q5. *Our* results are much better than *Wu et al.* in terms of the topology-related criteria (Q3 & Q4), and are of the compelling quality with the *GT* that are professionally-designed floorplans.

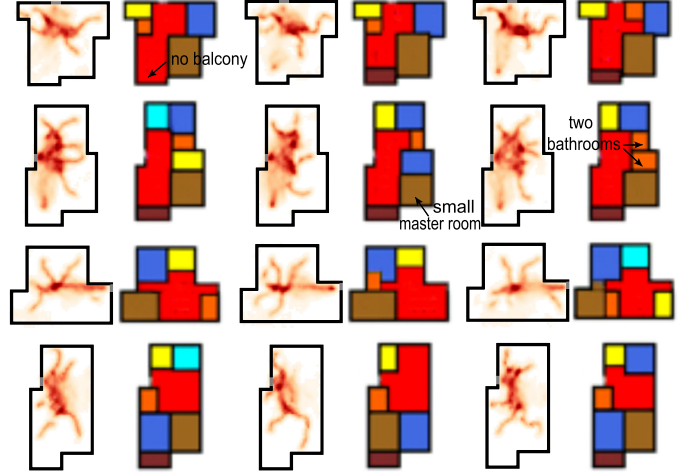


Fig. 13. Our approach can generate diverse floorplans in guidance by different human-activity maps created by the stochastic generator with dropouts.

results receive almost the same Elo ratings as those of *GT* for all Q1 to Q5, indicating that ActFloor-GAN can produce floorplans of the compelling quality with the professionally-designed floorplans.

6.4 Diverse Floorplans

Our approach can generate diverse floorplans by using different human-activity maps as guidance, which are prepared either by the automatic approach with a generative network or by the semi-automatic approach with an interactive interface.

Fig. 13 presents diverse floorplans produced by the automatic approach. The generative network is able to produce stochastic human-activity maps using dropouts added to the first three layers of the decoder. Here, the building boundaries are added as a reference. Note that the generated human-activity maps are subtly different, yet the floorplans are rather diverse with different room numbers and types. However, the floorplan designs may not be ideal, such as bad designs without balconies, a small master room, and two bathrooms next to each other. The semi-automatic approach can overcome the deficiency by allowing users to manipulate the human-activity maps on demand.

Fig. 14 presents diverse floorplans that can be derived from a single building boundary using the interactive interface (see Fig. 6). For the same building boundary, the generated floorplans possess different properties, including the number of rooms, room positions and sizes, room types, and room adjacency relations, and they are different from the ground truths from RPLAN. The human-activity maps are generated through our user interface by a user with no architecture background. Not just being diverse, our results may also be more reasonable than real-world floorplans. As shown in the last two rows of Fig. 14, the floorplans from the RPLAN dataset have no master room but several second rooms, which are unnatural for common residential buildings. Floorplans produced by our approach, instead, all have a master room.

Fig. 15 illustrates the effects of applying different interactions to the furniture on the human-activity maps and floorplans. In Fig. 15a, the washing machine is changed to a toilet. Similar human-activity maps are generated like the original

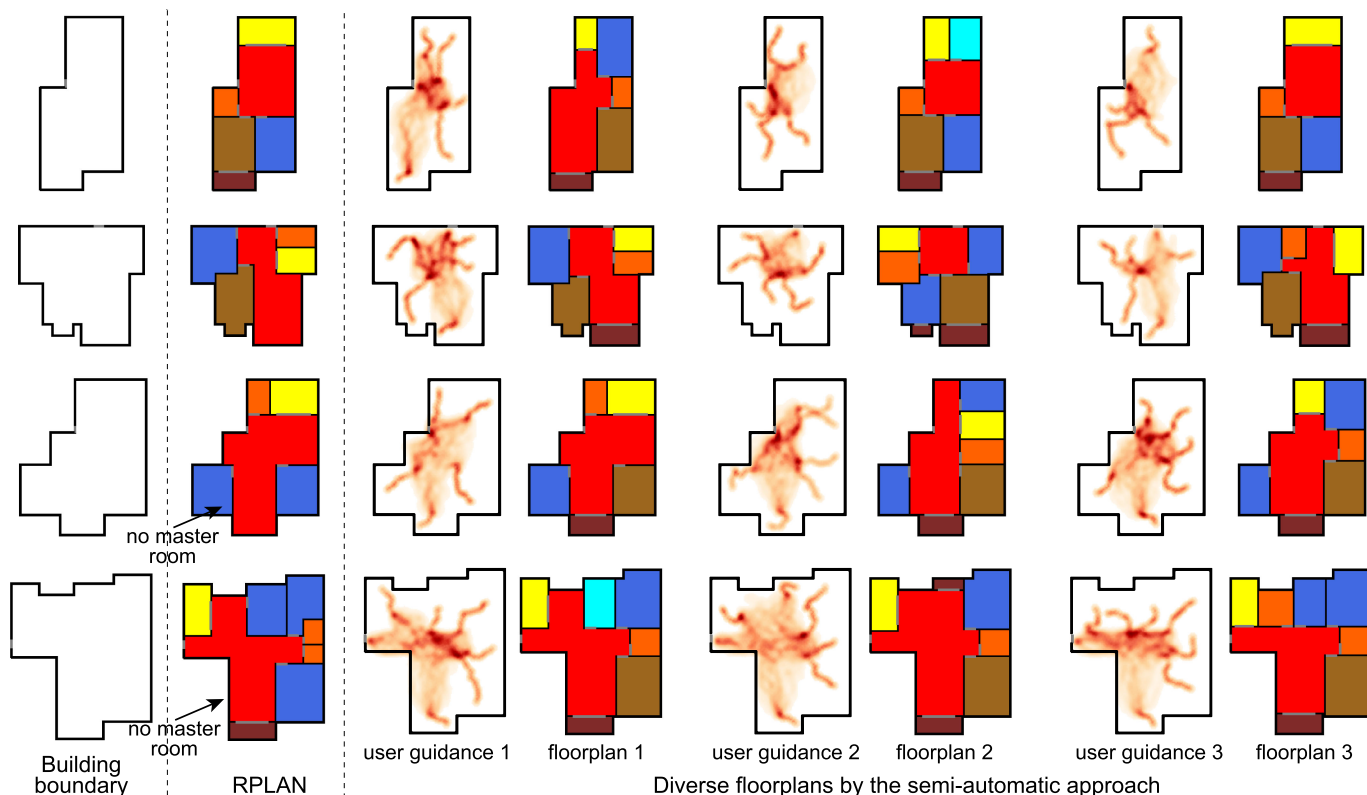


Fig. 14. Our approach can generate diverse and high-quality human-centric floorplans for the same input building boundary, thanks to the semi-automatic approach that can produce different human-activity maps based on user demands.

ones, and the floorplans have few changes. In Fig. 15b, the washing machine is removed, leading to minimal activities in the area. In this case, the balcony area in the original floorplan is merged with the living room. In Fig. 15c, the toilet position is adjusted, and the corresponding shape and size of the toilet and the neighboring rooms are adjusted. The results suggest that the human-activity map carries spatial information for room geometry guidance but no semantic information for inferring the room types.

The alternative approach of Wu *et al.* [51] by default can only generate one single floorplan for an input building boundary. To support the generation of diverse floorplans, they incorporate a probability distribution of predicted rooms, which, however, may not produce optimal floorplans. Similar to our approach, Graph2Plan [16] provides a user interface that allows users to query and select a room topology

graph, which supports the generation of diverse floorplans. However, the retrieved graph needs to be adjusted to fit the building boundary, which requires domain knowledge to generate fine floorplans. Nevertheless, even satisfactory room topology may not sufficiently capture the human spatial behavior and experience [10].

6.5 Discussion

Quality. Fig. 16 presents examples of *GT* and our floorplans presented in the evaluation with architects presented in Section 6.2. In Fig. 16 (left), our floorplan receives a lower rating than *GT*, though the layouts are almost the same. The only difference is that our floorplan lacks an extra balcony outside the master room. Even though the master room produced by our method is bigger, the interviewed architects explained that a master room with a balcony is typically more welcomed in practice. In Fig. 16 (right), our floorplan receives a higher rating than *GT*. In our floorplan, the balcony and kitchen are directly connected through a living room, whilst those in *GT* are obstructed by a bathroom and a

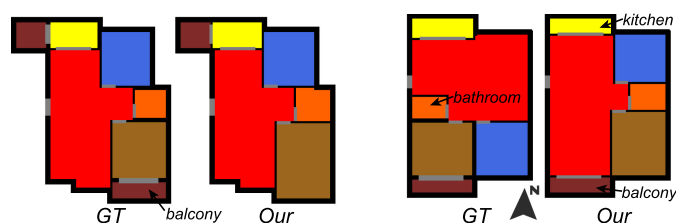


Fig. 16. Examples of *GT* and *Our* floorplans presented in the user evaluation by architects. *Our* floorplan receives a lower rating on the left, and a higher rating on the right than *GT*.

Fig. 15. Effects of different interactions applied to the furniture on human-activity maps and floorplans.

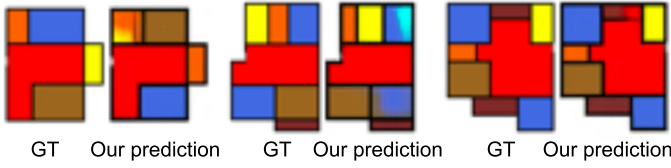


Fig. 17. Examples of noisy ActFloor-GAN predictions.

master room. Our floorplan is a transparent layout that wind can easily blow through from south to north and vice versa, which is much preferred by the architects.

Efficiency. Runtime is another important criterion for floorplan generation, as long delays can cause a bad user experience. Our approach takes less than 50ms to predict and generate a piecewise floorplan from an input boundary, including around 30ms on generating the human-activity map using the automatic approach, and around 16ms for ActFloor-GAN module. In contrast, Graph2Plan [16] takes about 178ms and Wu *et al.* [51] require a total of about 4s for the network to process the input and generate a result. Compared with the existing methods, our approach has an advantage in terms of time efficiency.

Limitations. Fig. 17 shows some noisy predictions generated by ActFloor-GAN. The left-hand side shows noisy kitchen and wall predictions for pixels in the upper-left corner, which should be a bathroom as shown in the GT floorplan. We notice that kitchen and bathroom pixels may be mistakenly predicted by ActFloor-GAN. This is probably because kitchens and bathrooms in the RPLAN dataset are similar in terms of room size and position. In the middle, there are noisy master room, second room, and study room predictions. These three room types can be collectively referred to as bedroom, and it is rather difficult for a network to learn the slight difference between them. In fact, Graph2Plan [16] treats all these three room types as bedroom. However, floorplan design in practice needs to separate these room types, since the placement of balcony and bathroom often depends on the position of the master room. On the right, predictions for the upper balcony are distracted by noisy predictions of the living room. This is probably because there are two balconies directly connected to the living room as shown in the GT floorplan, which is not common in the RPLAN dataset. The network model is not able to catch such abnormal cases.

The human-activity map reflects room function and improves the prediction accuracy than vanilla models such as Pix2Pix and CycleGAN. The more accurate prediction on pixel-wise room types also eases the vectorization process. Besides, the activity map reflects the intensity of human activities in the rooms. We find that the pixel of highest activity density in each room is highly correlated with position of the door, and use the information to place interior doors (see Fig. 8c). In comparison, the current model of Graph2Plan [16] using room topology is not able to predict doors. Yet, further information, such as visibility and accessibility [3], which better indicate room function, may help the network make noise-free predictions.

Another limitation of this work is the lack of model interpretability. In Fig. 15, we have shown that changes made to human-activity maps can lead to diverse floorplans, and Fig. 9 shows that the quality of final floorplans is highly affected by the synthesized human-activity maps. However,

variations of the floorplans upon changes made to the human-activity maps are not always predictable. Operations like removing furniture make big changes to human-activity maps, and ActFloor-GAN can predict a floorplan without the room. Yet, small changes made by moving furniture or changing furniture types may not always lead to floorplans on desire. Nevertheless, this is regarded as a general problem for existing deep learning models. Many efforts have been devoted to this direction. We would like to incorporate emerging techniques to improve the interpretability of ActFloor-GAN in the near future.

7 CONCLUSION AND FUTURE WORK

We presented ActFloor-GAN, a new deep framework for automated floorplan design. Unlike existing deep-learning-based approaches that try to directly learn the geometric or topological properties of floorplans, we propose to tackle the problem from a new perspective, by leveraging the human-activity map as guidance for network training. The benefit of introducing the human-activity map is prominent. First, to incorporate the human-activity map in guiding the learning process, we re-formulate the cycle-consistent constraints in the generative networks. We train ActFloor-GAN on the real-world floorplan dataset RPLAN, and conduct experiments showing that the guidance by the human-activity map and the cycle-consistent constraints can produce more accurate network predictions. Second, the human-activity map reflects resident interaction with the environment, so the generated floorplans by our approach are both geometrically plausible and topologically reasonable. As evaluated by five architects, the generated floorplans satisfy the requirement of being human-centric, and of the compelling quality with the professionally-designed ones. Third, the human-activity map depends on the room layout and furniture locations, so different human-activity maps can be derived by manipulating the furniture placement using a user interface. Correspondingly, we can generate diverse floorplans for the same given building boundary.

There are several promising directions for future work. Though the human-activity map improves the overall performance, its contributions to network prediction vary upon the room type. As shown in the failure cases, ActFloor-GAN may mistakenly mix kitchen and bathroom, and also the master room, second room, and study room. This is probably because our representation of the human-activity map is essentially a probability map of human spatial behavior. The collaborating architect suggests utilizing other architectural metrics that can encode further details on the human-environment interactions, which have been extensively studied in the space syntax [13]. We envision more accurate network predictions with these additional metrics. Moreover, we would like to extend the current framework from residential buildings to other scenarios like office space and public space design, which shall consider substantially more people movements in the planning stage [14].

ACKNOWLEDGMENTS

The authors wish to thank the anonymous reviewers for their valuable comments.

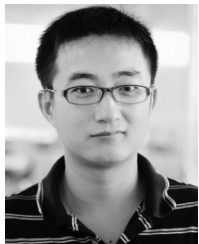
REFERENCES

- [1] S. A. Arvin and D. H. House, "Modeling architectural design objectives in physically based space planning," *Autom. Construction*, vol. 11, no. 2, pp. 213–225, 2002.
- [2] F. Bao, D.-M. Yan, N. J. Mitra, and P. Wonka, "Generating and exploring good building layouts," *ACM Trans. Graph.*, vol. 32, no. 4, pp. 1–10, 2013.
- [3] G. Berseth et al., "Interactive architectural design with diverse solution exploration," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 1, pp. 111–124, Jan. 2021.
- [4] Z. Bylinskii et al., "Learning visual importance for graphic designs and data visualizations," in *Proc. Annu. ACM Symp. User Interface Softw. Technol.*, 2017, pp. 57–69.
- [5] S. Chaillou, "AI & architecture: Towards a new approach," Master thesis, Graduate School Des., Harvard Univ., 2019.
- [6] Q. Chen, Q. Wu, R. Tang, Y. Wang, S. Wang, and M. Tan, "Intelligent home 3D: Automatic 3D-house design from linguistic descriptions only," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12625–12634.
- [7] X. Chen, W. Zeng, Y. Lin, H. M. Al-manee, J. Roberts, and R. Chang, "Composition and configuration patterns in multiple-view visualizations," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 2, pp. 1514–1524, Feb. 2021.
- [8] L. Corbusier and F. Etchells, *Vers Une Architecture. Towards a New Architecture*. Translated from the Thirteenth French Edition with an Introduction by Frederick Etchells. London, U.K.: John Rodker, 1927.
- [9] T. Feng, L.-F. Yu, S.-K. Yeung, K. Yin, and K. Zhou, "Crowd-driven mid-scale layout design," *ACM Trans. Graph.*, vol. 35, no. 4, pp. 1–14, 2016.
- [10] G. Franz and J. M. Wiener, "From space syntax to space semantics: A behaviorally and perceptually oriented methodology for the efficient description of the geometry and topology of environments," *Environ. Plan. B*, vol. 35, no. 4, pp. 574–592, 2008.
- [11] I. J. Goodfellow et al., "Generative adversarial nets," in *Proc. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
- [12] H. Hertzberger, *Lessons for Students in Architecture*. Rotterdam, The Netherlands: 010 Publishers, 1991.
- [13] B. Hillier, A. Leaman, P. Stansall, and M. Bedford, "Space syntax," *Environ. Plan. B: Plan. Des.*, vol. 3, no. 2, pp. 147–185, 1976.
- [14] B. Hillier, A. Penn, J. Hanson, T. Grajewski, and J. Xu, "Natural movement: Or, configuration and attraction in urban pedestrian movement," *Environ. Plan. B: Plan. Des.*, vol. 20, no. 1, pp. 29–66, 1993.
- [15] K. Hu, S. Yoon, V. Pavlovic, P. Faloutsos, and M. Kapadia, "Predicting crowd egress and environment relationships to support building design optimization," *Comput. Graph.*, vol. 88, pp. 83–96, 2020.
- [16] R. Hu, Z. Huang, Y. Tang, O. van Kaick, H. Zhang, and H. Huang, "Graph2plan: Learning floorplan generation from layout graphs," *ACM Trans. Graph.*, vol. 39, no. 4, pp. 118:11–18:14, 2020.
- [17] P. Isola, J. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5967–5976.
- [18] M. Jenkinson, P. Bannister, M. Brady, and S. Smith, "Improved optimization for the robust and accurate linear registration and motion correction of brain images," *Neuroimage*, vol. 17, no. 2, pp. 825–841, 2002.
- [19] J. Johnson, A. Alahi, and F.-F. Li, "Perceptual losses for real-time style transfer and super-resolution," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 694–711.
- [20] A. Kalervo, J. Ylioinas, M. Häikiö, A. Karhu, and J. Kannala, "CubiCasa5K: A dataset and an improved multi-task model for floorplan image analysis," in *Proc. Scand. Conf. Image Anal.*, 2019, pp. 28–40.
- [21] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of Gans for improved quality, stability, and variation," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–26.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–15.
- [23] Kim Young Ook, *Spatial Configuration, Spatial Cognition and Spatial Behaviour: The role of architectural intelligibility in shaping spatial experience*. Ph.D. Thesis, Univ. London, Univ. College London, London, U. K., 1999.
- [24] S. M. Lavalle, "Rapidly-exploring random trees: A new tool for path planning," *Comput. Sci. Dept., Iowa State Univ., Tech. Rep.* 98-11, 1998.
- [25] C. Ledig et al., "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 105–114.
- [26] C. Lee et al., "GUIComp: A GUI design assistant with real-time, multi-faceted feedback," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2020, pp. 1–13.
- [27] P. H. Levin, "Use of graphs to decide the optimum layout of buildings," *Architect*, vol. 14, pp. 809–815, 1964.
- [28] C. Li and M. Wand, "Precomputed real-time texture synthesis with markovian generative adversarial networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 702–716.
- [29] J. Li, K. Xu, S. Chaudhuri, E. Yumer, H. Zhang, and L. Guibas, "Grass: Generative recursive autoencoders for shape structures," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–14, 2017.
- [30] J. Li, J. Yang, A. Hertzmann, J. Zhang, and T. Xu, "LayoutGAN: Synthesizing graphic layouts with vector-wireframe adversarial networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 7, pp. 2388–2399, Jul. 2021.
- [31] J. Li, J. Yang, J. Zhang, C. Liu, C. Wang, and T. Xu, "Attribute-conditioned layout GAN for automatic graphic design," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 10, pp. 4039–4048, Oct. 2021.
- [32] M. Li et al., "GRAINS: Generative recursive autoencoders for indoor scenes," *ACM Trans. Graph.*, vol. 38, no. 2, pp. 1–16, 2019.
- [33] C. Liu, J. Wu, P. Kohli, and Y. Furukawa, "Raster-to-vector: Revisiting floorplan transformation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2214–2222.
- [34] Leonardo Benevolo, *History of Modern Architecture, Vol. 1: The Tradition of Modern Architecture*. Cambridge, MA, USA: MIT Press, 1977.
- [35] F. Maes, A. Collignon, D. Vandermeulen, G. Marchal, and P. Suetens, "Multimodality image registration by maximization of mutual information," *IEEE Trans. Med. Imag.*, vol. 16, no. 2, pp. 187–198, Apr. 1997.
- [36] P. Merrell, E. Schkufza, and V. Koltun, "Computer-generated residential building layouts," *ACM Trans. Graph.*, vol. 29, no. 6, pp. 1–12, 2010.
- [37] J. Michalek, R. Choudhary, and P. Papalambros, "Architectural layout design optimization," *Eng. Optim.*, vol. 34, no. 5, pp. 461–484, 2002.
- [38] Ming-Kuei Hu, "Visual pattern recognition by moment invariants," *IRE Trans. Inf. Theory*, vol. 8, no. 2, pp. 179–187, 1962.
- [39] M. Mirza and S. Osindero, "Conditional generative adversarial nets," *CoRR*, abs/1411.1784, 2014.
- [40] N. Nauata, K.-H. Chang, C.-Y. Cheng, G. Mori, and Y. Furukawa, "House-GAN: Relational generative adversarial networks for graph-constrained house layout generation," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 162–177.
- [41] A. G. Patil, O. Ben-Eliezer, O. Perel, and H. Averbuch-Elor, "Read: Recursive autoencoders for document layout generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2020, pp. 544–545.
- [42] N. Pelechano, J. M. Allbeck, and N. I. Badler, "Controlling individual agents in high-density crowd simulation," in *Proc. ACM SIGGRAPH/Eurograph. Symp. Comput. Animation*, 2007, pp. 99–108.
- [43] C.-H. Peng et al., "Computational network design from functional specifications," *ACM Trans. Graph.*, vol. 35, no. 4, pp. 1–12, 2016.
- [44] S. Qi, Y. Zhu, S. Huang, C. Jiang, and S.-C. Zhu, "Human-centric indoor scene synthesis using stochastic grammar," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 5899–5908.
- [45] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–16.
- [46] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 25–34, 1987.
- [47] E. Rodrigues, A. R. Gaspar, and A. Gomes, "An approach to the multi-level space allocation problem in architecture using a hybrid evolutionary technique," *Autom. Construction*, vol. 35, pp. 482–498, 2013.
- [48] A. Swearingin, M. Dontcheva, W. Li, J. Brandt, M. Dixon, and A. J. Ko, "Rewire: Interface design assistance from examples," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2018, pp. 1–12.
- [49] K. Wang, M. Savva, A. X. Chang, and D. Ritchie, "Deep convolutional priors for indoor scene synthesis," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–14, 2018.

- [50] W. Wu, L. Fan, L. Liu, and P. Wonka, "MIQP-based layout design for building interiors," *Comput. Graph. Forum*, vol. 37, no. 2, pp. 511–521, 2018.
- [51] W. Wu, X.-M. Fu, R. Tang, Y. Wang, Y.-H. Qi, and L. Liu, "Data-driven interior plan generation for residential buildings," *ACM Trans. Graph.*, vol. 38, no. 6, pp. 1–12, 2019.
- [52] Y.-L. Yang, J. Wang, E. Vouga, and P. Wonka, "Urban pattern: Layout design by hierarchical domain splitting," *ACM Trans. Graph.*, vol. 32, no. 6, pp. 1–12, 2013.
- [53] L.-F. Yu, S.-K. Yeung, C.-K. Tang, D. Terzopoulos, T. F. Chan, and S. J. Osher, "Make it home: Automatic optimization of furniture arrangement," *ACM Trans. Graph.*, vol. 30, no. 4, pp. 1–12, 2011.
- [54] W. Zeng, Q. Shen, Y. Jiang, and A. Telea, "Route-aware edge bundling for visualizing origin-destination trails in urban traffic," *Comput. Graph. Forum*, vol. 38, no. 3, pp. 581–593, 2019.
- [55] Z. Zeng, X. Li, Y. K. Yu, and C.-W. Fu, "Deep floor plan recognition using a multi-task network with room-boundary-guided attention," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 9096–9104.
- [56] X. Zheng, X. Qiao, Y. Cao, and R. W. H. Lau, "Content-aware generative modeling of graphic design layouts," *ACM Trans. Graph.*, vol. 38, no. 4, pp. 1–15, 2019.
- [57] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, "Unpaired image-to-image translation using cycle-consistent adversarial networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2223–2232.



Shidong Wang received the master's degree from the School of Computer Science and Technology, Shandong University. He was a visiting student with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences. His research interests include human-centric AI and computational design.



Wei Zeng (Member, IEEE) received the PhD degree in computer science from Nanyang Technological University. He is currently an assistant professor with the Hong Kong University of Science and Technology, Guangzhou. He was a senior researcher with Future Cities Laboratory, ETH Zurich and an associate researcher with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences. His research interests include visualization and visual analytics, computer graphics, AR or VR, and HCI.



Xi Chen is currently working toward the master's degree with the Shenzhen Institute of Advanced Technology, Chinese Academy of Sciences and with the University of Chinese Academy of Sciences. Her research interests include data visualization and human computer interaction.



Yu Ye received the PhD degree from The University of Hong Kong. He is currently an associate professor with the Department of Architecture, College of Architecture and Urban Planning, Tongji University, China. He was a postdoctoral researcher with Future Cities Laboratory, ETH Zurich. His research focuses on computational urban design utilizing multi-sourced urban data and newly emerged analytical techniques.



Yu Qiao (Senior Member, IEEE) is currently a professor with the Shenzhen Institute of Advanced Technology, Chinese Academy of Science and the director of Institute of Advanced Computing and Digital Engineering. His research interests include computer vision, deep learning, and bioinformation. He was the program chair of IEEE ICIST 2014. He was the recipient of the first prize of Guangdong technological invention Award and Jiaxi Lv young researcher Award from the Chinese Academy of Sciences.



Chi-Wing Fu (Member, IEEE) is currently a professor with the Chinese University of Hong Kong. His research interests include computation fabrication, point cloud processing, 3D computer vision, user interaction, and data visualization. He was the co-chair of SIGGRAPH ASIA 2016's Technical Brief and Poster program, an associate editor for *IEEE Computer Graphics & Applications* and *Computer Graphics Forum*, a panel member in SIGGRAPH 2019 Doctoral Consortium, and a program committee member in various research conferences, including SIGGRAPH Asia Technical Brief, SIGGRAPH Asia Emerging Technology, IEEE visualization, CVPR, IEEE VR, VRST, Pacific Graphics, and GMP.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**